

## Ressource Développement Web S3

### TD n°4 : classes, objets, interfaces

*Tout le code produit doit être conforme au standard PSR-1.  
Les types des arguments et les types de retour doivent être précisés.*

On souhaite créer une application web pour un service de diffusion de musique en ligne. L'application manipule donc principalement des pistes audio de différents types (piste d'album, podcast) ainsi que des listes de pistes (albums, playlists, liste d'attente).

#### Exercice 1

Construire une classe nommée **AlbumTrack** pour représenter les pistes audio d'un album. Chaque piste est décrite par les propriétés suivantes, correspondant pour la plupart aux tags ID3v2 habituellement inclus dans des fichiers audio : titre, artiste, album, année, numéro de piste (dans l'album), genre, durée (en secondes), nom du fichier audio.

- pour l'instant tous les attributs sont publics,
- prévoir un constructeur recevant uniquement le **titre de la piste** et le **chemin du fichier audio**, le **nom de l'album**, et le **numéro de piste** dans l'album,
- programmer une méthode pour transformer l'objet en une chaîne de caractères. On appellera la méthode `__toString()` et on utilise la fonction php `json_encode()` combinée avec la fonction `get_object_vars()` pour produire une chaîne de caractère au format json.
- nommer le fichier `AlbumTrack.php`

#### Exercice 2

Construire un programme qui utilise cette classe. Faites impérativement un fichier séparé (principe PSR-1) et utilisez `require_once` pour charger la définition de la classe dans ce programme.

Ce programme :

- crée deux pistes et valorise leurs attributs,
- affiche les informations de chaque piste en accédant aux différents attributs, et en utilisant les constructions et fonctions `echo`, `print`, `printf`. Examiner la documentation PHP pour comprendre les différences entre ces 3 instructions.
- affiche les objets complets en un seul appel, en utilisant la fonction `__toString()` (par exemple `echo $t1->__toString();`), puis sans utiliser cette méthode (`echo $t1 ;`) ; tester avec `echo` puis `print_r` puis `var_dump`

#### Exercice 3

On souhaite maintenant gérer l'affichage HTML d'une piste audio d'un album. A cette fin, on va créer la classe **AlbumTrackRenderer** dont le rôle sera de gérer les différents modes d'affichage d'une piste (par exemple mode "compact" pour un affichage en liste, mode "long" pour un affichage en plein écran). Cette classe présente les caractéristiques suivantes :

- le constructeur reçoit un objet de type `AlbumTrack`
- la conversion en html est réalisée par la méthode `render(int $selector): string` qui reçoit un sélecteur permettant de choisir le mode d'affichage et retourne un fragment de code html correspondant à l'affichage de la piste (penser à utiliser la balise html `<audio>`),
- la conversion effective est réalisée pour chaque mode par une méthode privée qui est appelée par la méthode `render` selon le cas.

Cette classe s'utilise donc de la manière suivante :

```
$t = new AlbumTrack("thriller", "thriller.flac") ;  
$r = new AlbumTrackRenderer( $t );
```

```
print $r->render( 1 ) ;    // 1 = mode compact
```

Compléter le programme principal pour qu'il affiche une piste en mode compact et en mode long.

#### **Exercice 4**

On souhaite maintenant ajouter la possibilité de gérer des podcasts dans notre application. Un podcast est une piste audio caractérisée par les propriétés suivantes : titre, auteur, date, genre, durée et nom de fichier audio. Un podcast est créé par un constructeur recevant le titre du podcast et le chemin vers le fichier audio.

- identifier les propriétés et méthodes communes entre un podcast et une piste audio d'un album. Créer en conséquence la classe **AudioTrack** qui regroupe ces caractéristiques communes et propose un constructeur,
- modifier la classe **AlbumTrack** pour qu'elle hérite d'**AudioTrack**. Le constructeur pour la classe **AlbumTrack** doit conserver la même signature mais doit utiliser le constructeur hérité de **AudioTrack**.
- créer la classe **PodcastTrack** qui hérite d'**AudioTrack**.
- compléter le programme principal pour créer un podcast.

#### **Exercice 5**

On souhaite maintenant afficher des podcasts, sur le même principe que des pistes audio d'album. Il va donc falloir créer la classe **PodcastRenderer** sur le même modèle que la classe **AlbumTrackRenderer**.

- Pour assurer une certaine compatibilité entre ces classes, on crée l'interface **Renderer** qui contient le profil de la méthode **render()** ainsi que deux constantes correspondant à des valeurs possibles pour les sélecteurs de mode d'affichage : **COMPACT** et **LONG**,
- transformer la classe **AlbumTrackRenderer** pour qu'elle implante l'interface **Renderer**,
- programmer la classe **PodcastRenderer** qui implante **Renderer** et permet de produire le code html d'affichage d'un podcast en mode compact et en mode long.
- Afficher un podcast dans votre programme principal.

#### **Exercice 6**

Refactorer les classes **AlbumTrackRenderer** et **PodcastTrackRenderer** en factorisant les parties communes dans une classe abstraite **AudioTrackRenderer**. La méthode **render()** est programmée dans cette classe en appliquant le patron "template de méthode".